

Конвеєри Jenkins

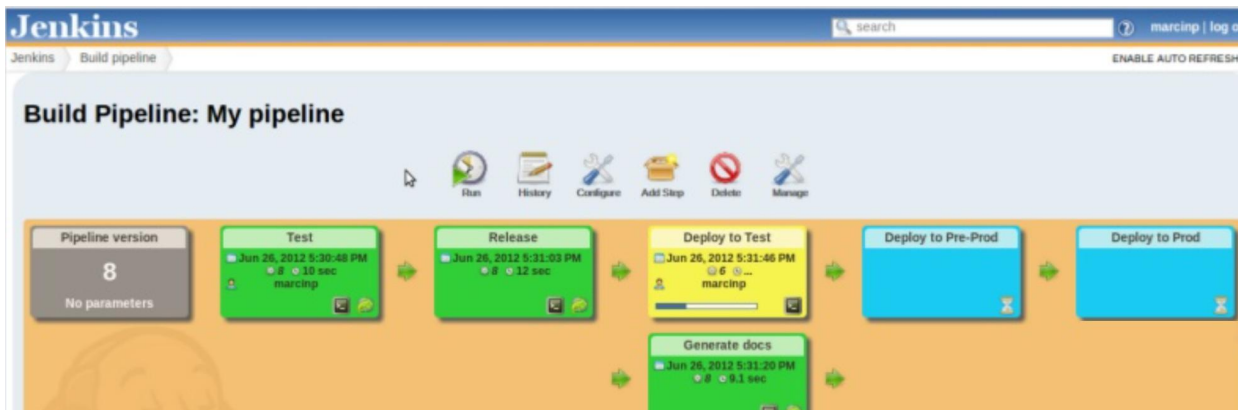
Чому Jenkins

- Створений проект з відкритим кодом
- Локальна установка
- Тисячі плагінів
- Інтегрується з багатьма інструментами/сервісами
- Спеціально для CI/CD



Історія CI/CD з Jenkins

- Завдання, що виконуються далі
- Завдання А запускає завдання В запускає завдання С запускає завдання D запускає...
- Якщо помилка в одному завданні, помилка всюди
- Перегляд конвеєра збірки



Конфігурація як код

- Визначте завдання/конвеєри як код
 - Уникайте наведення й натискання
 - У контролі версій
 - Може працювати із застосунком
 - Краще масштабується
- Приклад: .travis.yml (з TravisCI)
- Подібно до мови GitlabCI: служби php: - redis-server before_script: - сценарій встановлення композитора: - ./bin/phpunit -c ... сповіщення: slack: ...

```
language: php

services:
- redis-server

before_script:
- composer install

script:
- ./bin/phpunit -c ...

notifications:
  slack:
...

```

Керовані кодом підходи в Jenkins

- Конструктор завдань Jenkins
 - На основі Python / YAML, з проекту OpenStack
- Плагін Job DSL
 - Groovy DSL! (наприклад, надсилайте запит Github API і створюйте завдання для всіх філій)
 - Велика річ!
 - Але створює окремі завдання

```
job('my-project-main') { scm {  
    git('https://github.com/...')  
}  
triggers {  
    scm('H/15 * * * *')  
}  
publishers {  
    downstream('my-project-unit')  
}  
}
```

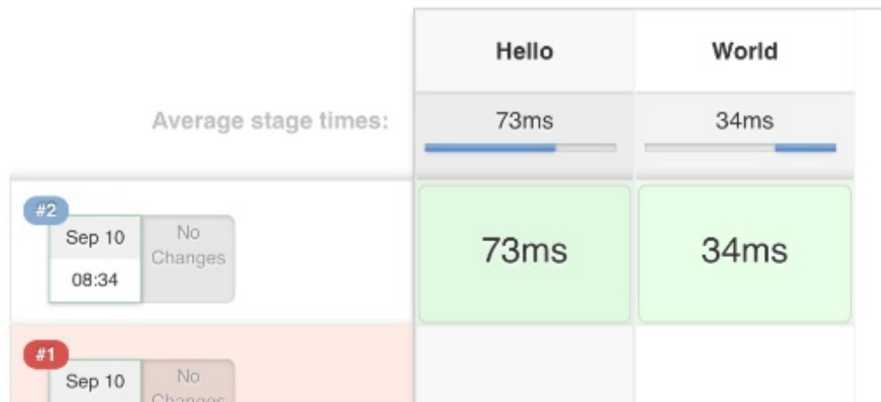
```
job('my-project-main') {  
    scm {  
        git('https://github.com/...')  
    }  
    triggers {  
        scm('H/15 * * * *')  
    }  
    publishers {  
        downstream('my-project-unit')  
    }  
}
```



Плагіни для конвеєрів Jenkins

Плагіни для конвеєрів Jenkins

- Весь набір плагінів (10+), відкритий на початку цього року
- Поставляється з Jenkins 2.0
- Раніше комерційно доступний CloudBees під назвою Workflow
- Визначає конвеєр як код (знову Groovy DSL)



```
stage("Hello") {  
    echo "*Hello*"  
}  
stage("World") {  
    echo "*World*"  
}
```

Приклади конвеєрів



Console Output

```
Started by user anonymous
[Pipeline] node
Running on master in /var/lib/jenkins/jobs/T3CM/jobs/Example/workspace
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] sh
[workspace] Running shell script
+ echo Could run composer install now
Could run composer install now
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Unit)
[Pipeline] sh
[workspace] Running shell script
+ echo Could run phpunit now
Could run phpunit now
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
```

```
node {
    stage("Build") {
        sh "echo Could run 'composer install' now"
    }
    stage("Unit") {
        sh "echo Could run 'phpunit' now"
    }
    // ...
}
```


Виконання конвеєра

- крок вузла виділяє слот виконавця («важкий виконавець»)
 - Як і інші завдання Jenkins
 - Фільтрувати вузол за мітками (тобто `node('php7')`, `node('master')`)
 - Потрібно для складних завдань
 - Уникайте багатьох послідовних розподілів (уповільнює прогрес конвеєра)
- Код поза вузлом
 - Легковаговий виконавець
 - Запуск на master, «безкоштовно»
- Виконання конвеєра переживає перезавантаження Jenkins (CPS)

Етапи конвеєрного DSL

- Запуск через оболонку
 - *nix systems: sh
 - `sh("make"), sh("rm -rf /")`
 - Windows systems: bat
 - `bat("C:Program Files...")`
- SCM
 - `checkout("https://github.com/..git")`
- Обробка файлів
 - `readFile("my.config")`
 - `writeFile(file: "README.md", text: "Hello World")`
 - `fileExists`

Етапи конвеєрного DSL (2)

- Запустити інше завдання:

```
build("jobname")
build("jobname", wait: false, propagate: false)
Build(job: 'jobname', parameters: [
    [$class: 'StringParameterValue', name: 'target', value: target]
    [$class: 'ListSubversionTagsParameterValue', name: 'release', tag: release],
    [$class: 'BooleanParameterValue', name: 'update_composer', value:
    update_composer.to_boolean()]])
```

- Не буду вдаватися далі в подробиці

Етапи конвеєрного DSL (3)

- Скопіювати робочу область до наступного виконавця

```
stage("build") {  
    node {  
        sh("make")  
        stash("my-workspace")  
    }  
}  
stage("release") {  
    node('magic-release-tool') {  
        unstash("my-workspace")  
        sh("release")  
    }  
}
```

Етапи конвеєрного DSL (4)

- Укажіть користувацькі змінні середовища

```
withEnv(['MYTOOL_HOME=/usr/local/mytool']) {  
    sh '$MYTOOL_HOME/bin/start'  
}
```

```
env.FOO = "bar"  
sh 'echo $BAR'
```

- Використовуйте змінні, надані Jenkins

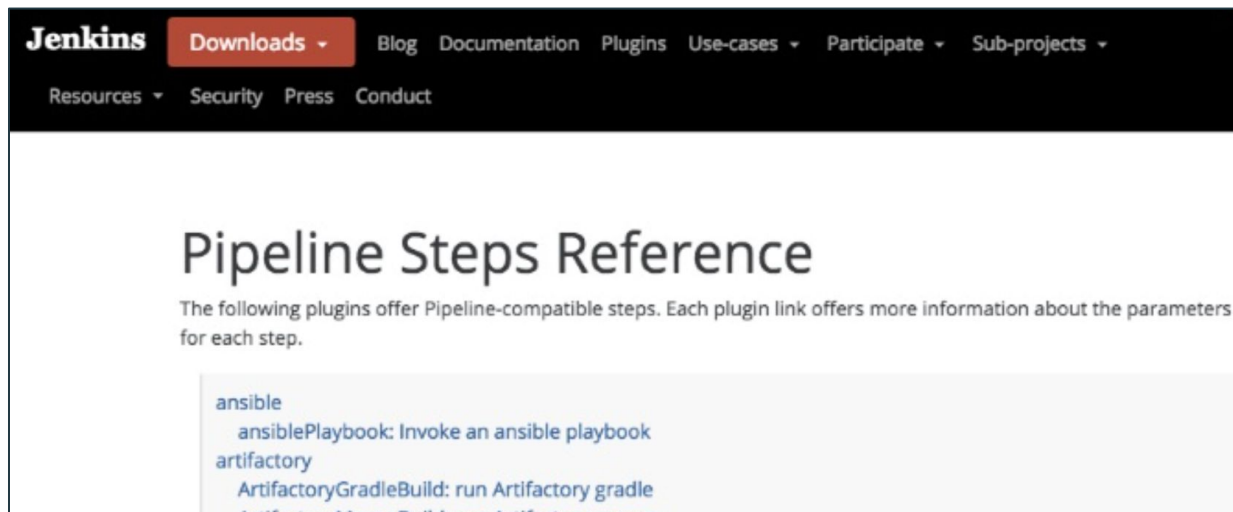
```
sh 'echo $BUILD_NUMBER > version.txt'
```

- Глобальні змінні Groovy

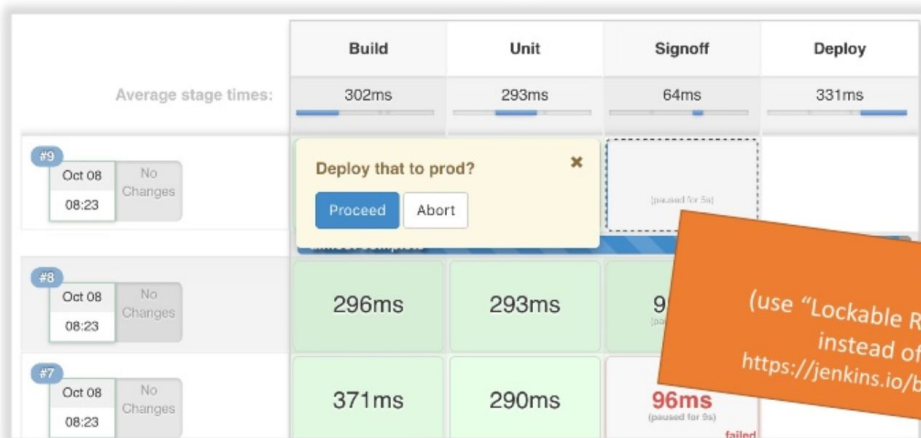
```
currentBuild.result = 'FAILED'
```

Додаткові етапи в пайплайн

- Плагіни забезпечують додаткові дії
- Кроки, доступні в цьому екземплярі Jenkins чер  Pipeline Syntax
- Онлайн довідка: <https://jenkins.io/doc/pipeline/steps/>



Приклад: підтвердження розгортання вручну



Deprecated!
(use "Lockable Resources" plugin and its lock step instead of concurrency parameter)
<https://jenkins.io/blog/2016/10/16/stage-lock-milestone/>

```
stage("Signoff") {  
    input("Deploy that to prod?")  
    milestone()  
}  
stage(name: "Deploy", concurrency: 1) { node {..} }
```

Docker

- Виконуйте завдання збирання в контейнерах Docker
- Немає необхідності встановлювати програмне забезпечення на головний/підлеглий пристрій Jenkins
- Використовуйте кілька версій одного інструменту
- Контейнери можуть бути наявними або побудованими на вимогу
- .. та Kubernetes

```
stage("run in docker") {  
    node {  
        withDockerContainer("php:7-fpm") {  
            sh "php -v"  
        }  
    }  
}
```


Вирізаний редактор і документи

- Бо перші кроки важкі..
- Автоматично створена документація DSL (Синтаксис конвеєра → Довідник по кроках)

Steps

Sample Step `writeFile: Write file to workspace`

File path in workspace

README.txt

Text to write

Hello World

Character encoding

Generate Groovy

`writeFile file: 'README.txt', text: 'Hello World'`

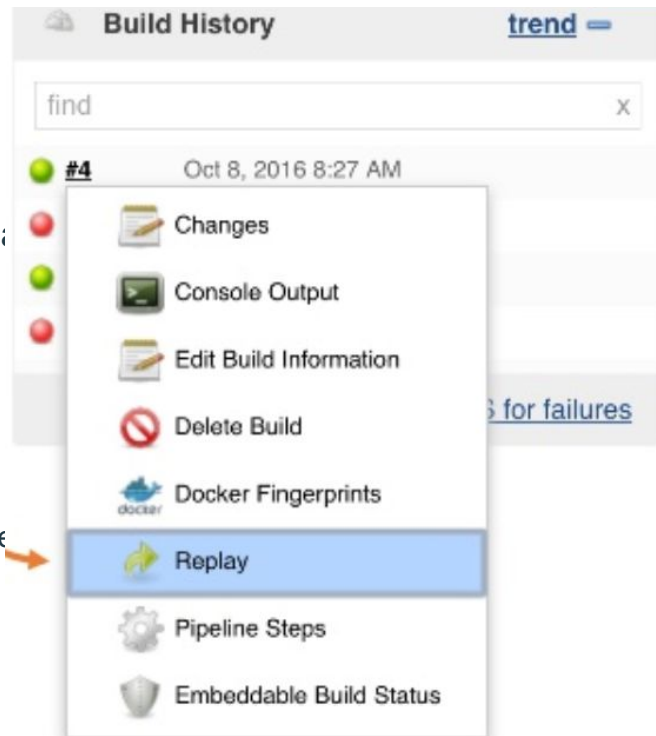


Куди подіти цю
конфігурацію?



Конфігурація конвеєра

- Вставте код у конфігурацію завдання (добре для тестування)
- Створюйте конвеєри за допомогою JobDSL
- Закріпіть це у своєму репо
 - Файл під назвою Jenkinsfile
 - Він розвивається разом із додатком
 - Це версії
 - Кожен може прочитати (і змінити) його
 - Ви можете викинути свого майстра Jenkins у будь-який момент



Плагіни багатогалузевої та організаційної папки

- Сканує повну організацію GitHub/Bitbucket для Jenkinsfile
- Ініціюється Webhook та/або запускається періодично
- Автоматично додає конвеєрні завдання для кожного репо/філії/PR



The screenshot displays the Jenkins web interface. The top navigation bar shows 'Jenkins' and the current path 'Jenkins > TYPO3's Chef Cookbooks'. On the left sidebar, there are links for 'Up', 'Status', 'Re-scan Organization', 'People', 'Build History', and 'GitHub'. Below these are sections for 'Build Queue' (showing 'No builds in the queue.') and 'Build Executor Status' (showing two idle executors). The main content area is titled 'TYPO3's Chef Cookbooks' and shows the 'Folder name: TYPO3-cookbooks'. A 'Repositories' table lists the configured repositories.

S	W	Repository ↓	Description
		backuppoc	Chef cookbook for BackupPC
		backuppoc = develop	
		gerrit	Chef cookbook for the Gerrit review system.
		gerrit = develop	
		gerrit = master	

DRY: Глобальна бібліотека Jenkins

- Надає спільну функціональність, доступну для всіх завдань
- Зберігається на головному сервері Jenkins і доступне для всіх агентів
- Доступно через сервер Git, інтегрований у Jenkins
- Можна завантажити з віддалених сховищ Git, указаних у глобальній конфігурації та Jenkinsfile

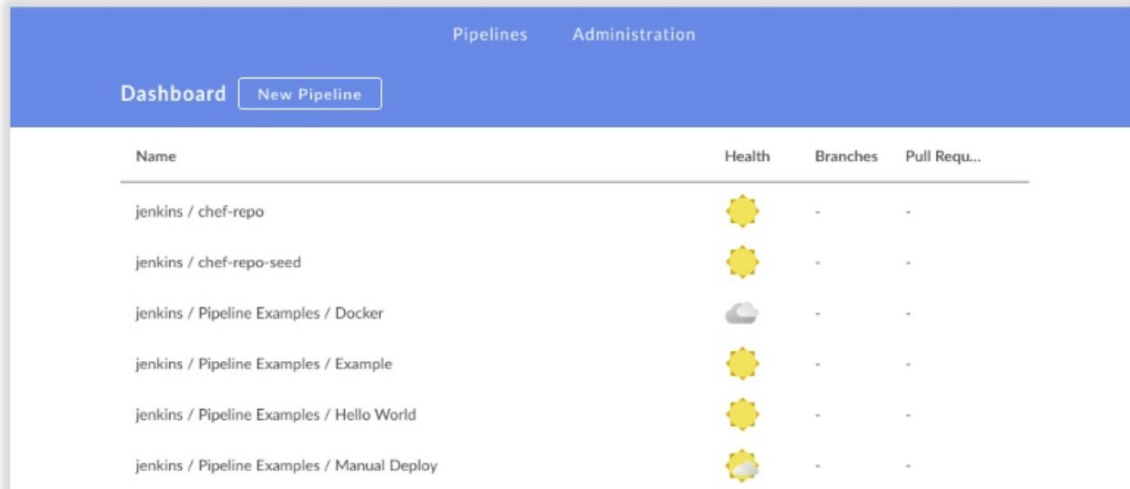
```
@Library("https://github.com/..")  
node { deployToProd() }
```



Blue Ocean та інші НОВИНИ

Blue Ocean

- Спеціально для конвеєрних плагінів

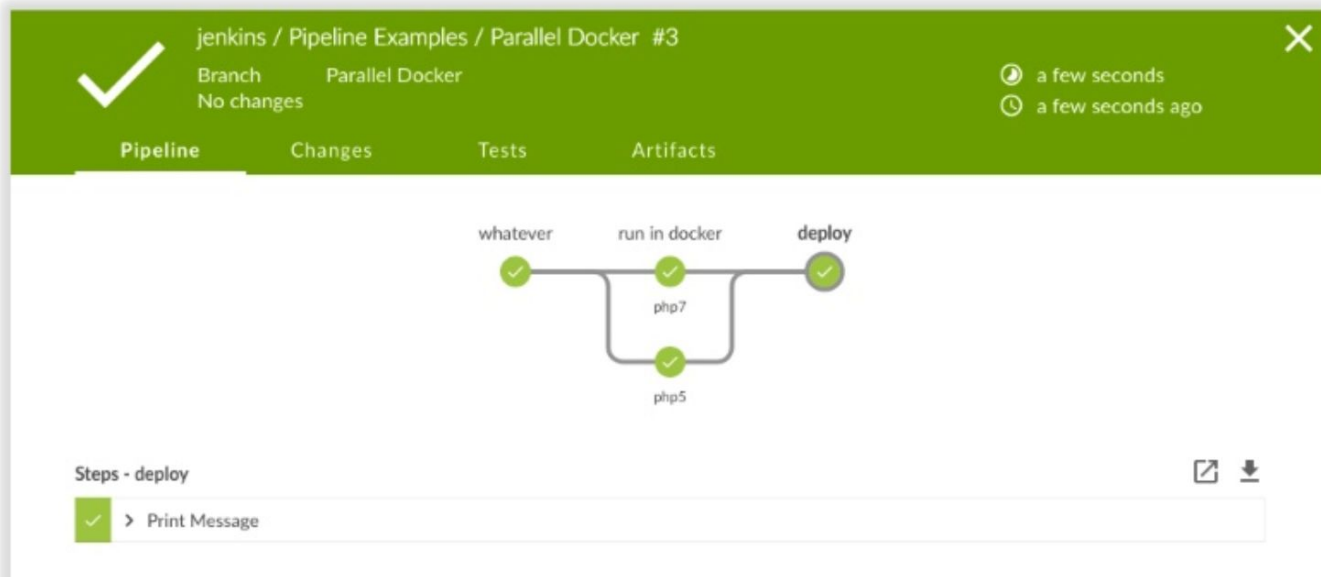


The screenshot shows the Jenkins Blue Ocean dashboard. At the top, there are tabs for 'Pipelines' and 'Administration'. Below these, there is a 'Dashboard' label and a 'New Pipeline' button. The main content is a table with four columns: 'Name', 'Health', 'Branches', and 'Pull Requ...'. The table lists several pipelines, including 'jenkins / chef-repo', 'jenkins / chef-repo-seed', 'jenkins / Pipeline Examples / Docker', 'jenkins / Pipeline Examples / Example', 'jenkins / Pipeline Examples / Hello World', and 'jenkins / Pipeline Examples / Manual Deploy'. The 'Health' column uses icons to represent the status: a yellow gear for 'OK', a grey cloud for 'Unstable', and a yellow gear with a red exclamation mark for 'Failing'.

Name	Health	Branches	Pull Requ...
jenkins / chef-repo		-	-
jenkins / chef-repo-seed		-	-
jenkins / Pipeline Examples / Docker		-	-
jenkins / Pipeline Examples / Example		-	-
jenkins / Pipeline Examples / Hello World		-	-
jenkins / Pipeline Examples / Manual Deploy		-	-

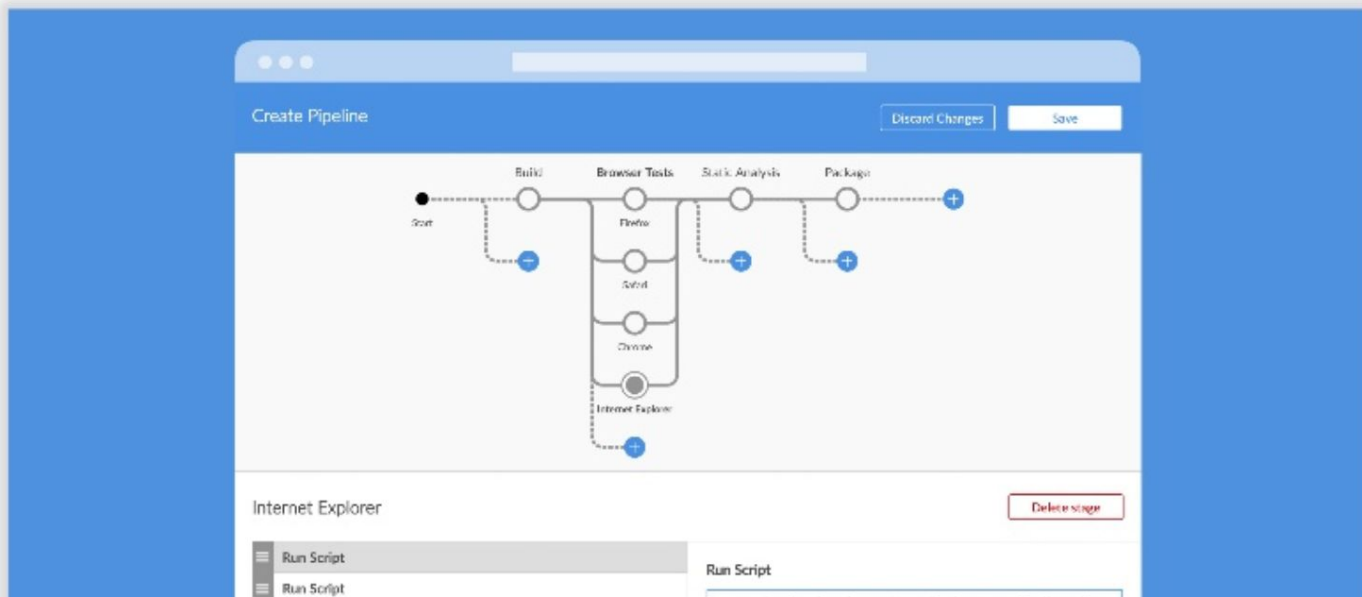
Blue Ocean

- Спеціально для конвеєрних плагінів



Редактор конвеєра*

* Нещодавно анонсований, ще не доступний



Декларативні конвеєри

- Декларативний підхід вирішує деякі проблеми (обробка відмов, кроки після створення)

```
pipeline {  
  agent docker:'node:6.3'  
  stages {  
    stage('build') {  
      sh '..'  
    }  
    stage('test') {  
      sh 'npm test'  
    }  
  }  
  postBuild {  
    always {  
      sh 'echo "This will always run"'  
    }  
    failure {  
      sh 'echo "This will run only if failed"'    }  
  }  
}
```

Підсумок

- Набір конвеєрів Jenkins модернізує свої можливості CI/CD
- CloudBees, Inc. дуже активно просуває розвиток
- Багато плагінів Jenkins уже сумісні
- Конвеєр, визначений як код
- Версійний
- Не псує роботу Jenkins
- Спільне використання коду
- Автоматизоване створення конвеєра на основі API GitHub/Bitbucket
- Blue Ocean оновлює інтерфейс Jenkins
- Ще залишаються деякі недоліки (обробка помилок, часті зміни)